

College Management System

1. Create Database

```
CREATE DATABASE CollegeDB;
```

2. Use Database

```
USE CollegeDB;
```

3. Create Tables

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50),  
    HOD_Name VARCHAR(50)  
);
```

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DOB DATE,  
    Gender VARCHAR(10),  
    Phone VARCHAR(15),  
    Email VARCHAR(50),  
    Dept_ID INT,  
    Address VARCHAR(100),  
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
);
```

```
CREATE TABLE Faculty (  
    Faculty_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Dept_ID INT,  
    Designation VARCHAR(50),  
    Phone VARCHAR(15),  
    Email VARCHAR(50),  
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
);
```

```
CREATE TABLE Course (  
    Course_ID INT PRIMARY KEY,  
    Course_Name VARCHAR(50),  
    Credits INT,  
    Dept_ID INT,  
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
);
```

```
CREATE TABLE Enrollment (  
    Enroll_ID INT PRIMARY KEY,  
    Student_ID INT,  
    Course_ID INT,  
    Semester INT,  
    Year INT,  
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),  
    FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)  
);
```

```
CREATE TABLE Marks (  
    Mark_ID INT PRIMARY KEY,  
    Student_ID INT,  
    Course_ID INT,  
    Internal INT,  
    External INT,  
    Total INT,  
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),  
    FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)  
);
```

```
CREATE TABLE Attendance (  
    Attendance_ID INT PRIMARY KEY,  
    Student_ID INT,  
    Course_ID INT,  
    Date DATE,  
    Status VARCHAR(10),  
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),  
    FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)  
);
```

4. Insert Sample Data

INSERT INTO Department VALUES

(1, 'CSE', 'Dr. Rao'),
(2, 'ECE', 'Dr. Sharma');

INSERT INTO Student VALUES

(101, 'Arjun', '2003-05-12', 'Male', '9876543210', 'arjun@gmail.com', 1, 'Udupi'),
(102, 'Priya', '2004-08-20', 'Female', '9123456780', 'priya@gmail.com', 2, 'Mangalore');

INSERT INTO Course VALUES

(201, 'DBMS', 4, 1),
(202, 'CN', 4, 1);

INSERT INTO Faculty VALUES

(301, 'Dr. Anil Kumar', 1, 'Professor', '9876501111', 'anil@college.com'),
(302, 'Dr. Meena Rao', 2, 'Associate Professor', '9876502222', 'meena@college.com');

INSERT INTO Enrollment VALUES

(401, 101, 201, 3, 2025),
(402, 102, 202, 3, 2025),
(403, 101, 202, 3, 2025);

INSERT INTO Marks VALUES

(501, 101, 201, 25, 65, 90),
(502, 102, 202, 28, 60, 88),
(503, 101, 202, 30, 58, 88);

INSERT INTO Attendance VALUES

(601, 101, 201, '2025-06-01', 'Present'),
(602, 101, 201, '2025-06-02', 'Absent'),
(603, 102, 202, '2025-06-01', 'Present'),
(604, 102, 202, '2025-06-02', 'Present'),
(605, 101, 202, '2025-06-03', 'Present');

5. Queries

SELECT * FROM Student;

```
mysql> SELECT * FROM Student;
```

Student_ID	Name	DOB	Gender	Phone	Email	Dept_ID	Address
101	Arjun	2003-05-12	Male	9876543210	arjun@gmail.com	1	Udupi
102	Priya	2004-08-20	Female	9123456780	priya@gmail.com	2	Mangalore

2 rows in set (0.00 sec)

SELECT * FROM Faculty;

```
mysql> SELECT * FROM Faculty;
```

Faculty_ID	Name	Dept_ID	Designation	Phone	Email
301	Dr. Anil Kumar	1	Professor	9876501111	anil@college.com
302	Dr. Meena Rao	2	Associate Professor	9876502222	meena@college.com

```
2 rows in set (0.00 sec)
```

SELECT * FROM Course;

```
mysql> SELECT * FROM Course;
```

Course_ID	Course_Name	Credits	Dept_ID
201	DBMS	4	1
202	CN	4	1

```
2 rows in set (0.00 sec)
```

SELECT * FROM Enrollment;

```
mysql> SELECT * FROM Enrollment;
```

Enroll_ID	Student_ID	Course_ID	Semester	Year
401	101	201	3	2025
402	102	202	3	2025
403	101	202	3	2025

```
3 rows in set (0.00 sec)
```

SELECT * FROM Marks;

```
mysql> SELECT * FROM Marks;
```

Mark_ID	Student_ID	Course_ID	Internal	External	Total
501	101	201	25	65	90
502	102	202	28	60	88
503	101	202	30	58	88

```
3 rows in set (0.00 sec)
```

SELECT * FROM Attendance;

```
mysql> SELECT * FROM Attendance;
```

Attendance_ID	Student_ID	Course_ID	Date	Status
601	101	201	2025-06-01	Present
602	101	201	2025-06-02	Absent
603	102	202	2025-06-01	Present
604	102	202	2025-06-02	Present
605	101	202	2025-06-03	Present

```
5 rows in set (0.00 sec)
```

SELECT * FROM Department;

```
mysql> SELECT * FROM Department;
```

Dept_ID	Dept_Name	HOD_Name
1	CSE	Dr. Rao
2	ECE	Dr. Sharma

```
2 rows in set (0.00 sec)
```

```
SELECT S.Name, D.Dept_Name
FROM Student S
JOIN Department D ON S.Dept_ID = D.Dept_ID;
```

```
mysql> SELECT S.Name, D.Dept_Name
-> FROM Student S
-> JOIN Department D ON S.Dept_ID = D.Dept_ID;
```

Name	Dept_Name
Arjun	CSE
Priya	ECE

2 rows in set (0.00 sec)

```
SELECT S.Name, C.Course_Name
FROM Student S
JOIN Enrollment E ON S.Student_ID = E.Student_ID
JOIN Course C ON E.Course_ID = C.Course_ID;
```

```
mysql> SELECT S.Name, C.Course_Name
-> FROM Student S
-> JOIN Enrollment E ON S.Student_ID = E.Student_ID
-> JOIN Course C ON E.Course_ID = C.Course_ID;
```

Name	Course_Name
Arjun	DBMS
Priya	CN
Arjun	CN

3 rows in set (0.00 sec)

```
SELECT Student_ID, COUNT(*) AS Total_Days,
SUM(CASE WHEN Status='Present' THEN 1 ELSE 0 END) AS Present_Days
FROM Attendance
GROUP BY Student_ID;
```

```
mysql> SELECT Student_ID, COUNT(*) AS Total_Days,
-> SUM(CASE WHEN Status='Present' THEN 1 ELSE 0 END) AS Present_Days
-> FROM Attendance
-> GROUP BY Student_ID;
```

Student_ID	Total_Days	Present_Days
101	3	2
102	2	2

2 rows in set (0.00 sec)

```
SELECT F.Name, D.Dept_Name, F.Designation
FROM Faculty F
JOIN Department D ON F.Dept_ID = D.Dept_ID;
```

```
mysql> SELECT F.Name, D.Dept_Name, F.Designation
-> FROM Faculty F
-> JOIN Department D ON F.Dept_ID = D.Dept_ID;
```

Name	Dept_Name	Designation
Dr. Anil Kumar	CSE	Professor
Dr. Meena Rao	ECE	Associate Professor

2 rows in set (0.00 sec)

```
SELECT Course_ID, COUNT(Student_ID) AS Total_Students
FROM Enrollment
GROUP BY Course_ID;
```

```
mysql> SELECT Course_ID, COUNT(Student_ID) AS Total_Students
-> FROM Enrollment
-> GROUP BY Course_ID;
```

Course_ID	Total_Students
201	1
202	2

2 rows in set (0.00 sec)

```
SELECT Student_ID, SUM(Total) AS Total_Marks
FROM Marks
GROUP BY Student_ID;
```

```
mysql> SELECT Student_ID, SUM(Total) AS Total_Marks
-> FROM Marks
-> GROUP BY Student_ID;
```

Student_ID	Total_Marks
101	178
102	88

2 rows in set (0.00 sec)

```
SELECT Student_ID, MAX(Total) AS Highest_Mark
FROM Marks
GROUP BY Student_ID;
```

```
mysql> SELECT Student_ID, MAX(Total) AS Highest_Mark
-> FROM Marks
-> GROUP BY Student_ID;
```

Student_ID	Highest_Mark
101	90
102	88

2 rows in set (0.03 sec)

```
SELECT Student_ID,
COUNT(*) AS Total_Days,
SUM(CASE WHEN Status='Present' THEN 1 ELSE 0 END) AS Present_Days
FROM Attendance
GROUP BY Student_ID;
```

```
mysql> SELECT Student_ID,
-> COUNT(*) AS Total_Days,
-> SUM(CASE WHEN Status='Present' THEN 1 ELSE 0 END) AS Present_Days
-> FROM Attendance
-> GROUP BY Student_ID;
```

Student_ID	Total_Days	Present_Days
101	3	2
102	2	2

2 rows in set (0.00 sec)

```
SELECT DISTINCT Student_ID
FROM Attendance
WHERE Status = 'Absent';
```

```
mysql> SELECT DISTINCT Student_ID
-> FROM Attendance
-> WHERE Status = 'Absent';
```

Student_ID
101

```
1 row in set (0.00 sec)
```

```
SELECT Course_ID, COUNT(*) AS Total_Students
FROM Enrollment
GROUP BY Course_ID
ORDER BY Total_Students DESC;
```

```
mysql> SELECT Course_ID, COUNT(*) AS Total_Students
-> FROM Enrollment
-> GROUP BY Course_ID
-> ORDER BY Total_Students DESC;
```

Course_ID	Total_Students
202	2
201	1

```
2 rows in set (0.00 sec)
```

```
SELECT S.Name, C.Course_Name
FROM Student S
JOIN Enrollment E ON S.Student_ID = E.Student_ID
JOIN Course C ON E.Course_ID = C.Course_ID
WHERE S.Name = 'Arjun';
```

```
mysql> SELECT S.Name, C.Course_Name
-> FROM Student S
-> JOIN Enrollment E ON S.Student_ID = E.Student_ID
-> JOIN Course C ON E.Course_ID = C.Course_ID
-> WHERE S.Name = 'Arjun';
```

Name	Course_Name
Arjun	DBMS
Arjun	CN

```
2 rows in set (0.00 sec)
```

6. Triggers

```
CREATE TRIGGER trg_calculate_total
BEFORE INSERT ON Marks
FOR EACH ROW
SET NEW.Total = NEW.Internal + NEW.External;
```

```
DELIMITER $$
```

```
CREATE TRIGGER check_attendance
BEFORE INSERT ON Attendance
FOR EACH ROW
BEGIN
    IF NEW.Status NOT IN ('Present', 'Absent') THEN
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Invalid status';
    END IF;
END$$
```

```
CREATE TRIGGER trg_check_marks
BEFORE INSERT ON Marks
FOR EACH ROW
BEGIN
    IF NEW.Internal < 0 OR NEW.External < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Marks cannot be negative';
    END IF;
END$$
```

```
CREATE TRIGGER trg_check_enrollment
BEFORE INSERT ON Attendance
FOR EACH ROW
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM Enrollment
        WHERE Student_ID = NEW.Student_ID
        AND Course_ID = NEW.Course_ID
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Student not enrolled in this course';
    END IF;
END$$
DELIMITER ;
```